

Task Scheduler

Main.c

```
/**
*****
 * @file    main.c
 * @author  Sagar More
 * @brief   Minimal RTOS Scheduler using SysTick + PendSV
*****
 */

#include <stdint.h>
#include "main.h"

/* ===== TASK PROTOTYPES ===== */
void task1_handler(void);
void task2_handler(void);
void task3_handler(void);
void task4_handler(void);

/* ===== SCHEDULER PROTOTYPES ===== */
void init_tasks_stack(void);
void init_systick(uint32_t tick_hz);
void switch_sp_to_psp(void);
void enable_faults(void);

/* ===== GLOBAL VARIABLES ===== */

/* PSP values of each task */
uint32_t psp_of_tasks[MAX_TASKS] =
{
    T1_STACK_START,
    T2_STACK_START,
    T3_STACK_START,
    T4_STACK_START
};

/* Task entry functions */
uint32_t task_handlers[MAX_TASKS];

/* Index of currently running task */
uint8_t current_task = 0;

/* Debug counters */
volatile uint32_t t1, t2, t3, t4;

/* ===== MAIN ===== */

int main(void)
{
    /* Enable MemManage, BusFault, UsageFault */
    enable_faults();
```

```

/* Initialize MSP for scheduler */
__asm volatile ("MSR MSP, %0" :: "r"(SCHED_STACK_START));

/* Register task handlers */
task_handlers[0] = (uint32_t)task1_handler;
task_handlers[1] = (uint32_t)task2_handler;
task_handlers[2] = (uint32_t)task3_handler;
task_handlers[3] = (uint32_t)task4_handler;

/* Initialize task stacks */
init_tasks_stack();

/* Start SysTick */
init_systick(TICK_HZ);

/* Switch from MSP to PSP */
switch_sp_to_psp();

/* Scheduler runs via interrupts */
while (1);
}

/* ===== SYSTICK HANDLER ===== */
/* Only triggers PendSV (no context switching here) */

void SysTick_Handler(void)
{
    /* Set PendSV pending bit */
    *(uint32_t *)0xE00ED04 |= (1 << 28);
}

/* ===== PENDSV HANDLER ===== */
/* Performs actual context switching */

__attribute__((naked)) void PendSV_Handler(void)
{
    /* Save current task context */
    __asm volatile ("MRS R0, PSP"); // Get PSP of current task
    __asm volatile ("STMDB R0!, {R4-R11}"); // Save R4-R11 on task
stack
    __asm volatile ("LDR R1, =psp_of_tasks"); // Base address of PSP
array
    __asm volatile ("LDR R2, =current_task"); // Address of current_task
    __asm volatile ("LDRB R3, [R2]"); // R3 = current_task
    __asm volatile ("STR R0, [R1, R3, LSL #2]"); // Save PSP value

    /* Select next task */
    __asm volatile ("ADD R3, R3, #1"); // current_task++
    __asm volatile ("CMP R3, #4"); // Compare with MAX_TASKS
    __asm volatile ("BNE next_task_ok");
    __asm volatile ("MOV R3, #0"); // Wrap to task 0
    __asm volatile ("next_task_ok:");
    __asm volatile ("STRB R3, [R2]"); // Update current_task

    /* Restore next task context */
    __asm volatile ("LDR R0, [R1, R3, LSL #2]"); // Load next task PSP
    __asm volatile ("LDmia R0!, {R4-R11}"); // Restore R4-R11
}

```

```

        __asm volatile ("MSR PSP, R0");           // Update PSP
        __asm volatile ("BX LR");                 // Return from
exception
}

/* ===== TASK STACK INITIALIZATION ===== */

void init_tasks_stack(void)
{
    uint32_t *psp;

    for (int i = 0; i < MAX_TASKS; i++)
    {
        /* Start from top of stack */
        psp = (uint32_t *)psp_of_tasks[i];

        *--psp = DUMMY_XPSR;           // xPSR (Thumb bit set)
        *--psp = task_handlers[i];     // PC (task entry)
        *--psp = 0xFFFFFFF0;          // LR (return to Thread mode,
PSP)
        *--psp = 0;                     // R12
        *--psp = 0;                     // R3
        *--psp = 0;                     // R2
        *--psp = 0;                     // R1
        *--psp = 0;                     // R0

        /* Manually stacked registers (R4â€”R11) */
        for (int j = 0; j < 8; j++)
        {
            *--psp = 0;
        }

        /* Save updated PSP */
        psp_of_tasks[i] = (uint32_t)psp;
    }
}

/* ===== SWITCH MSP TO PSP ===== */

__attribute__((naked)) void switch_sp_to_psp(void)
{
    __asm volatile ("LDR R0, =psp_of_tasks"); // Address of PSP array
    __asm volatile ("LDR R0, [R0]");          // PSP of task 0
    __asm volatile ("MSR PSP, R0");           // Load PSP

    __asm volatile ("MOV R0, #0x02");         // Use PSP in Thread mode
    __asm volatile ("MSR CONTROL, R0");
    __asm volatile ("ISB");                   // Instruction sync
    __asm volatile ("BX LR");
}

/* ===== SYSTICK INITIALIZATION ===== */

void init_systick(uint32_t tick_hz)
{
    uint32_t count = (SYSTICK_TIM_CLK / tick_hz) - 1;

    *(uint32_t *)0xE000E014 = count;          // Reload value

```

```

        *(uint32_t *)0xE000E010 =
            (1 << 2) |      // Processor clock
            (1 << 1) |      // SysTick interrupt enable
            (1 << 0);       // Enable SysTick
    }

/* ===== TASKS ===== */

void task1_handler(void)
{
    while (1) t1++;
}

void task2_handler(void)
{
    while (1) t2++;
}

void task3_handler(void)
{
    while (1) t3++;
}

void task4_handler(void)
{
    while (1) t4++;
}

/* ===== FAULT ENABLE ===== */

void enable_faults(void)
{
    uint32_t *SHCSR = (uint32_t *)0xE000ED24;

    *SHCSR |= (1 << 16); // MemManage
    *SHCSR |= (1 << 17); // BusFault
    *SHCSR |= (1 << 18); // UsageFault
}

```

Main.h

```
/*
 * main.h
 *
 * Created on: Jan 29, 2026
 * Author: Sagar More
 */

#ifndef MAIN_H_
#define MAIN_H_

#include <stdint.h>

/* Stack sizes */
#define SIZE_TASK_STACK 1024U
#define SIZE_SCEDULAR_STACK 1024U

/* SRAM */
#define SRAM_START 0x20000000U
#define SRAM_SIZE (128U * 1024U)
#define SRAM_END (SRAM_START + SRAM_SIZE)

/* Stack start addresses */
#define T1_STACK_START SRAM_END
#define T2_STACK_START (SRAM_END - (1U * SIZE_TASK_STACK))
#define T3_STACK_START (SRAM_END - (2U * SIZE_TASK_STACK))
#define T4_STACK_START (SRAM_END - (3U * SIZE_TASK_STACK))
#define SCHED_STACK_START (SRAM_END - (4U * SIZE_TASK_STACK))

/* SysTick */
#define TICK_HZ 1000U
#define HSI_CLK 16000000U
#define SYSTICK_TIM_CLK HSI_CLK

/* Scheduler */
#define MAX_TASKS 4U

/* CPU */
#define DUMMY_XPSR 0x01000000U /* Thumb bit */

#endif /* MAIN_H_ */
```

Output:

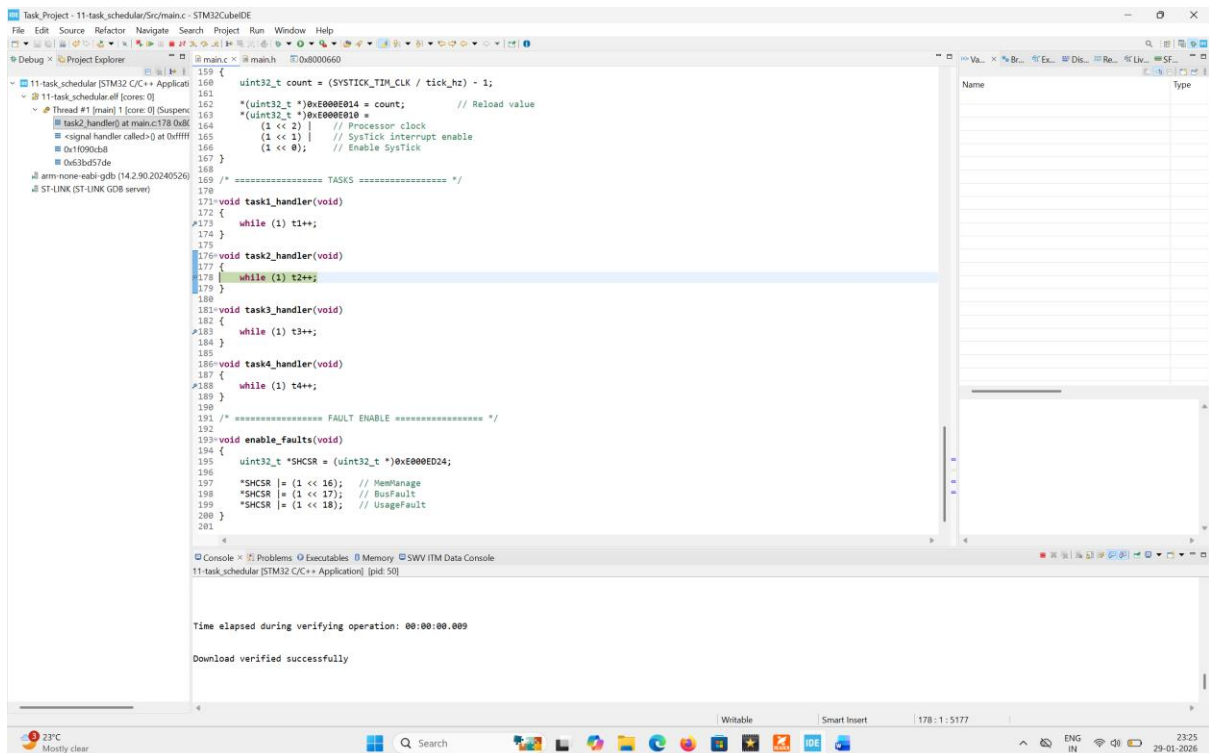


Figure 1: Task 1

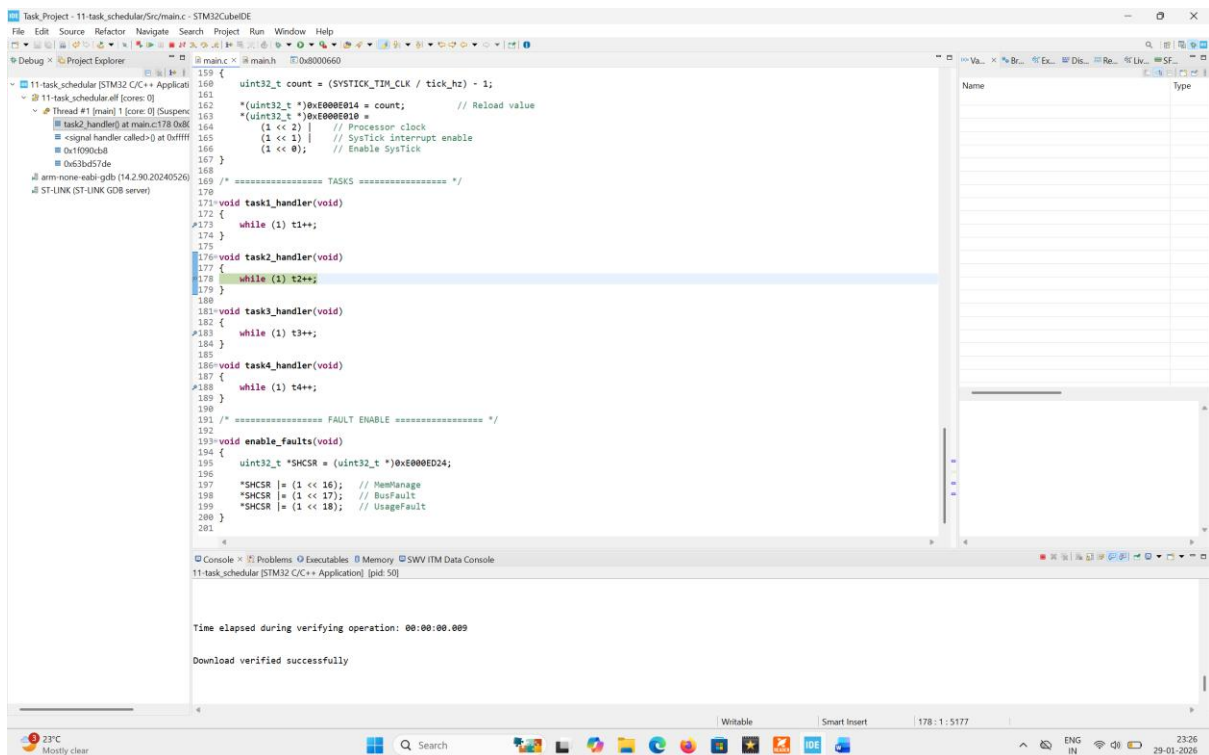


Figure 2: Task 2

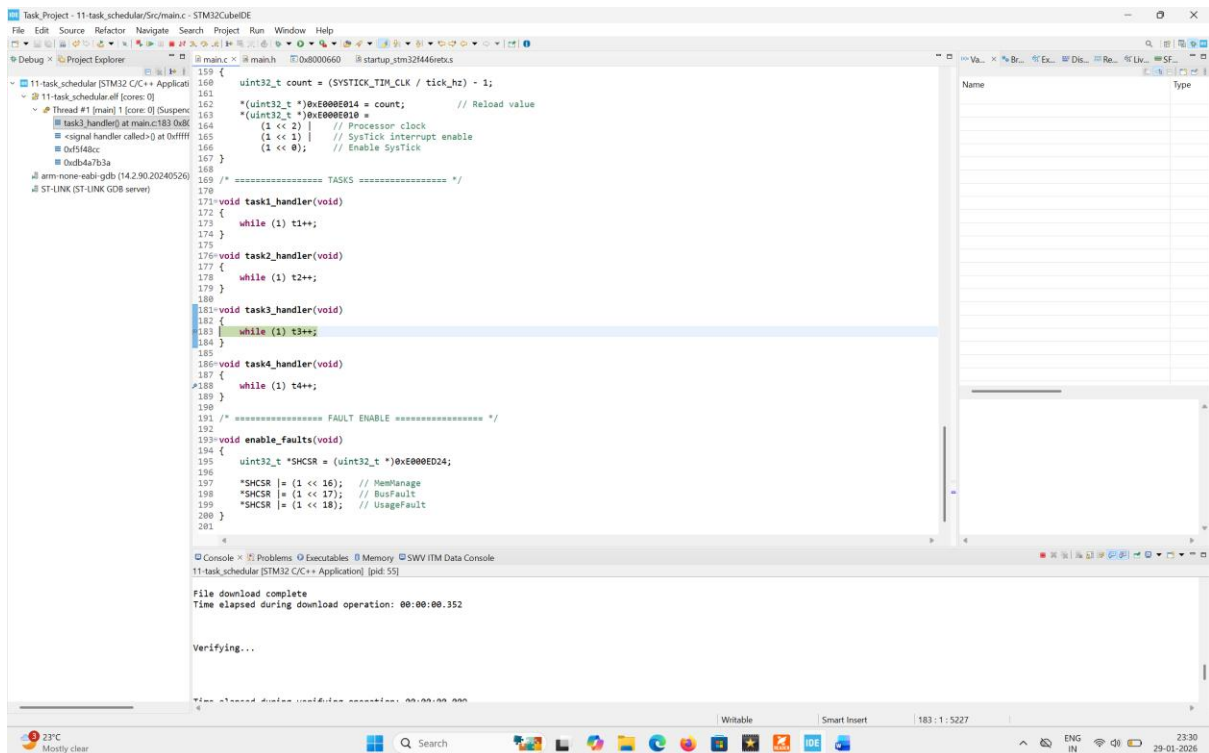


Figure 3: Task 3

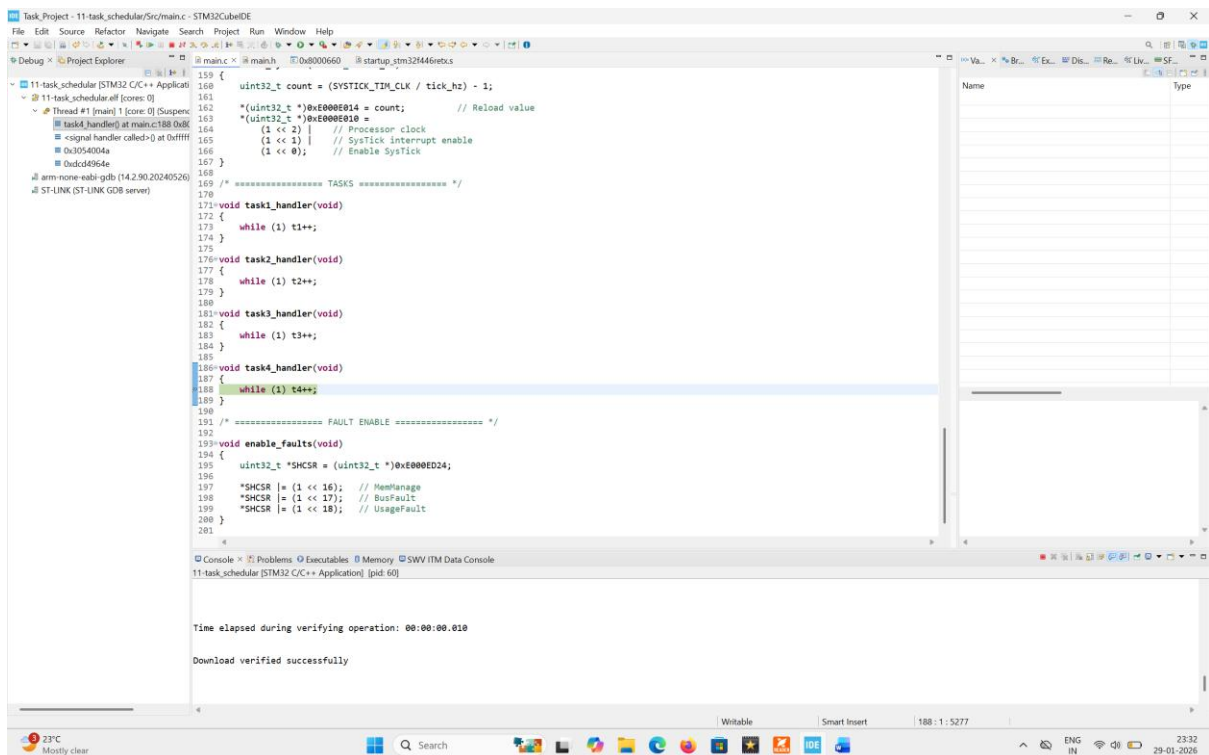


Figure 4: Task 4